

Habit Tracker App – Full Project Documentation

1. Project Overview

Habit Tracker is a modern web application that helps users build and maintain positive habits by tracking daily and weekly activities. The app focuses on motivation through **streaks**, **visual progress**, and **clear feedback**.

This project is designed to be: - Portfolio-ready - Scalable - Docker-friendly - Beginner-to-intermediate React friendly

2. Core Features

2.1 Habit Management

- Create new habits
- Edit habit details
- Delete habits
- Set habit frequency:
 - Daily
 - Weekly

2.2 Habit Tracking

- Mark habit as completed for a day
- Prevent duplicate completion for the same date
- Auto-reset weekly habits

2.3 Streak System 🔥

- Daily streak calculation
- Weekly streak calculation
- Longest streak record
- Streak breaks when a habit is missed

2.4 Progress Calendar 🌞

- Monthly calendar view
- Completed days highlighted
- Missed days shown visually
- Hover / click to view habit status

2.5 Progress Analytics

- Completion percentage

- Current streak vs best streak
- Weekly and monthly summaries

2.6 UX Enhancements

- Responsive design
 - Dark / light mode
 - Animations for streak milestones
-

3. Tech Stack

Frontend

- **React 18**
- **Vite** (fast build tool)
- **TypeScript** (recommended)
- **React Router** (page navigation)
- **Context API** (global state)
- **date-fns** (date utilities)
- **CSS Modules / Tailwind CSS** (styling)

State Management

- Context API + Reducer
- Optional upgrade: Zustand or Redux Toolkit

Persistence

- LocalStorage (Phase 1)
- Optional:
- Firebase Firestore
- Supabase
- REST API backend

DevOps

- Docker
 - Docker Compose
 - Nginx (production)
-

4. Application Pages

4.1 Home / Dashboard

- Today's habits

- Quick complete buttons
- Current streak summary

4.2 Habits Page

- List all habits
- Filter: Daily / Weekly
- Habit completion history

4.3 Calendar Page

- Monthly calendar
- Color-coded habit completion
- Click date to see details

4.4 Analytics Page

- Charts and stats
- Habit consistency trends

4.5 Settings

- Theme toggle
- Reset data
- Export habit data (JSON)

5. Data Model

Habit Model

```
Habit {  
  id: string;  
  name: string;  
  frequency: 'daily' | 'weekly';  
  createdAt: string;  
  completedDates: string[]; // ISO dates  
  currentStreak: number;  
  longestStreak: number;  
}
```

6. Streak Logic (Core Algorithm)

Daily Habit

- If completed today → streak +1
- If missed yesterday → streak resets

Weekly Habit

- One completion per week counts
- Missed week resets streak

Edge Cases

- Timezone-safe date comparison
 - Prevent double counting
-

7. Folder Structure

```
src/
├── components/
│   ├── HabitCard.tsx
│   ├── Calendar.tsx
│   └── StreakBadge.tsx
├── pages/
│   ├── Dashboard.tsx
│   ├── Habits.tsx
│   ├── CalendarView.tsx
│   └── Analytics.tsx
├── context/
│   ├── HabitContext.tsx
│   └── habitReducer.ts
├── hooks/
│   └── useHabits.ts
├── utils/
│   ├── dateUtils.ts
│   └── streakUtils.ts
└── styles/
```

```
|— App.tsx
|— main.tsx
```

8. Docker Setup

Dockerfile (Development)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev", "--", "--host"]
```

docker-compose.yml

```
version: "3.8"
services:
  habit-tracker:
    build: .
    ports:
      - "5173:5173"
    volumes:
      - ./app
      - /app/node_modules
```

9. Development Phases

Phase 1 – MVP

- Habit CRUD
- Daily tracking
- LocalStorage

Phase 2 – Visual Progress

- Calendar view
- Streak counter

Phase 3 – Analytics

- Charts
- Performance optimization

Phase 4 – Production

- Docker + Nginx
 - Environment configs
-

10. Testing Strategy

- Unit tests (Jest / Vitest)
 - Component tests (React Testing Library)
 - Manual edge-case testing for streak logic
-

11. Deployment Options

- Vercel (Frontend)
 - Netlify
 - Docker + VPS
 - AWS EC2 with Nginx
-

12. Resume Description (Ready-to-use)

Built a Habit Tracker web application using React and TypeScript, featuring daily and weekly habit tracking, streak calculation, and calendar-based progress visualization. Implemented Dockerized development environment and scalable state management.

13. Future Enhancements

- User authentication
 - Cloud sync
 - Push notifications
 - Mobile app (React Native)
 - Gamification (badges, levels)
-

14. Success Criteria

- Clean UX

- Accurate streak logic
- Zero date bugs
- Dockerized setup
- Resume-worthy codebase

🔥 This project demonstrates **frontend fundamentals**, **real-world logic**, and **production readiness**.